



Java Basics, JVM & Syntax

Prof. Ashwani Mishra

Assistant Professor
AIML Department



Content

1. Introduction to Java.....	3
2. JDK, JRE & JVM.....	8
3. Data Types & Variables.....	14
4. Control Flow & Programs.....	20
5. References... ..	25



What is Java?

- Java is a high-level, object-oriented programming language developed at Sun Microsystems.
- Programs compile to bytecode that runs on the Java Virtual Machine (JVM).
- The JVM makes Java platform independent: the same bytecode runs on Windows, Linux, and macOS.
- Java is widely used for desktop, web, mobile (Android), and enterprise applications.
- The language emphasizes readability, safety, and a rich standard class library.
- Parul University Semester 3 course 303105205 builds on these Java fundamentals.



History of Java

Java History Milestones

Year	Release	Highlight
1991	Oak Project	James Gosling at Sun Microsystems
1995	Java 1.0	Public release - Write Once, Run Anywhere
2006	Java SE 6	Scripting and web services support
2014	Java 8	Lambda expressions and Stream API
2021	Java 17 LTS	Long-term support for enterprise

Figure 1.2 Java History Timeline



Key Features of Java

Key Features of Java

Platform & Security

- Platform independent via JVM bytecode
- Automatic memory management (garbage collection)
- Strong type checking at compile time
- Built-in security manager and sandbox

Language & Ecosystem

- Pure object-oriented (except primitives)
- Rich standard library (java.* packages)
- Multithreading support in the language
- Huge open-source and enterprise ecosystem

Figure 1.3 Key Features of Java



Write Once, Run Anywhere

- Java source code (.java) is compiled once into bytecode (.class) by javac.
- Bytecode is not machine code; it is an intermediate format understood by any JVM.
- Each operating system provides its own JVM implementation for that platform.
- This model eliminates recompilation when moving programs between platforms.
- The slogan WORA captures this portability advantage.



Figure 1.4 Write Once, Run Anywhere



Your First Java Program

- Every Java application starts execution in the main method.
- The public static void main(String[] args) method is the program entry point.
- System.out.println prints text to the console followed by a new line.
- Class name must match the filename (HelloWorld.java contains class HelloWorld).
- Compile with javac, then run with java followed by the class name (no .class extension).

```
 HelloWorld.java

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
        System.out.println("Subject: OOP3 - Semester 3");
    }
}

// Save as HelloWorld.java
// Compiler: javac HelloWorld.java
// Run: java HelloWorld
```

Figure 1.5 HelloWorld.java



JDK, JRE and JVM Overview

JDK vs JRE vs JVM

Component	Contains	Used By
JDK	javac, jar, javadoc, JRE	Developers who write Java code
JRE	JVM + core libraries (rt.jar)	End users running Java apps
JVM	Class loader, bytecode verifier, GC	Every Java program at runtime
javac	Compiler in JDK only	Converts .java to .class bytecode

Figure 1.6 JDK vs JRE vs JVM



Understanding the Java Platform Stack

- The JDK (Java Development Kit) is the complete toolkit for writing Java programs.
- The JRE (Java Runtime Environment) includes the JVM plus core libraries needed at runtime.
- The JVM loads bytecode, verifies it, and executes it on the native operating system.
- Developers install the JDK; end users who only run programs need the JRE (or a bundled JRE).
- Modern JDK distributions often ship as a single package without a separate JRE.

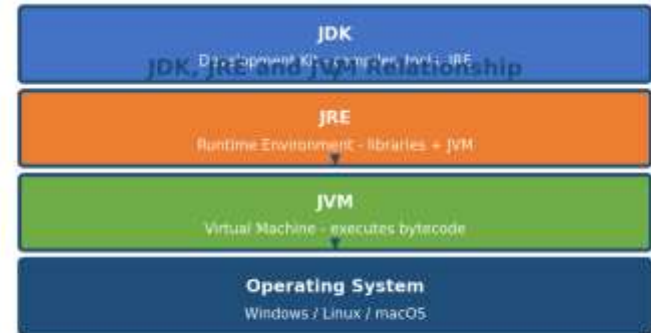


Figure 1.7 Java Platform Stack



Java Compilation Flow

Java Compilation and Execution Flow



Figure 1.8 Compilation and Execution Flow



From Source Code to Running Program

- Step 1: Developer writes human-readable Java source in a .java text file.
- Step 2: The javac compiler checks syntax and type rules, then emits .class bytecode files.
- Step 3: The JVM class loader reads .class files into memory when the program starts.
- Step 4: The bytecode verifier ensures the code cannot harm the host system.
- Step 5: The JVM interpreter or JIT compiler translates bytecode into native machine instructions.
- Step 6: The operating system executes native code while the JVM manages memory and threads.



JVM Memory Areas

JVM Memory Areas

Area	Stores	Lifecycle
Method Area	Class metadata, static variables	Shared across threads
Heap	Objects and instance variables	Garbage collected automatically
Stack	Method frames, local variables	Per thread, LIFO structure
PC Register	Current instruction address	One per thread
Native Method Stack	Native (C/C++) method calls	Platform dependent

Figure 1.10 JVM Memory Areas



How the JVM Manages Memory

- The Method Area stores class definitions, constant pool data, and static variables.
- The Heap is shared memory where all objects live; the garbage collector reclaims unused objects.
- Each thread gets its own Stack holding method call frames and local primitive variables.
- Stack memory is automatically freed when a method returns; heap objects persist until collected.
- StackOverflowError occurs when call depth exceeds stack limits; OutOfMemoryError affects the heap.
- Understanding heap vs stack helps explain variable scope, object lifetime, and performance.

JVM Memory Areas

Area	Stores	Lifecycle
Method Area	Class metadata, static variables	Shared across threads
Heap	Objects and instance variables	Garbage collected automatically
Stack	Method frames, local variables	Per thread, LIFO structure
PC Register	Current instruction address	One per thread
Native Method Stack	Native (C/C++) method calls	Platform dependent

Figure 1.11 JVM Memory Management



Primitive Data Types

Java Primitive Data Types

Type	Size	Range / Example
byte	1 byte	-128 to 127
short	2 bytes	-32,768 to 32,767
int	4 bytes	Most common for integers
long	8 bytes	Large whole numbers (suffix L)
float / double	4 / 8 bytes	Decimal numbers (suffix f / default)
char	2 bytes	Single Unicode character ('A')
boolean	1 bit (JVM)	true or false only

Figure 1.12 Primitive Data Types



Variables and Data Types in Practice

- Variables are named memory locations that hold values of a specific type.
- Java is statically typed: every variable must declare its type before use.
- Primitive types (int, double, char, boolean, etc.) store simple values directly.
- Reference types (String, arrays, objects) store a reference (address) to heap memory.
- Variable names must start with a letter or underscore and follow camelCase convention.
- Local variables must be initialized before reading; fields get default values (0, false, null).

Java Primitive Data Types

Type	Size	Range / Example
byte	1 byte	-128 to 127
short	2 bytes	-32,768 to 32,767
int	4 bytes	Most common for integers
long	8 bytes	Large whole numbers (suffix L)
float / double	4 / 8 bytes	Decimal numbers (suffix f / default)
char	2 bytes	Single Unicode character ('A')
boolean	1 bit (JVM)	true or false only

Figure 1.13 Variables and Types



Declaring and Using Variables

- The int type stores whole numbers; double stores floating-point decimal values.
- The char type holds a single character enclosed in single quotes.
- The boolean type represents logical true or false conditions in expressions.
- String is a reference type (class) used for text, enclosed in double quotes.
- The + operator concatenates strings: name + " scored " + percentage.
- System.out.println displays formatted output on the console for debugging and demos.

```
VariablesDemo.java

public class VariablesDemo {
    public static void main(String[] args) {
        int rollNo = 101;
        double percentage = 87.5;
        char grade = 'A';
        boolean passed = true;
        String name = "Amit Vaghanshi";

        System.out.println(name + " scored " + percentage + "%");
    }
}
```

Figure 1.14 Variable Declaration Example



Operators in Java

Java Operators Overview

Category	Operators	Example
Arithmetic	+ - * / %	a + b, a % b (remainder)
Relational	== != > < >= <=	age >= 18
Logical	&& !	(x > 0) && (y > 0)
Assignment	= += -= *=	count += 1
Increment	++ --	i++ (post), ++i (pre)
Ternary	? :	result = (a > b) ? a : b

Figure 1.15 Java Operators



Working with Operators

- Arithmetic operators perform math: addition, subtraction, multiplication, division, modulus.
- Relational operators compare values and produce a boolean result for decision making.
- Logical operators (&&, ||, !) combine boolean expressions in if and loop conditions.
- Assignment operators such as += update a variable using its current value (count += 1).
- Increment operators ++ and -- add or subtract one; prefix vs postfix affects expression value.
- The ternary operator (condition ? valueIfTrue : valueIfFalse) provides compact if-else logic.

Figure 1.16 Operator Categories



Reading Input with Scanner

- The `java.util.Scanner` class reads formatted input from the console (`System.in`).
- Create a Scanner with `new Scanner(System.in)` at the start of your program.
- `nextLine()` reads an entire line of text; `nextInt()`, `nextDouble()` read numeric tokens.
- Close the Scanner with `sc.close()` when finished to release system resources.

```
ScannerDemo.java

import java.util.Scanner;

public class ScannerDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter your name:");
        String name = sc.nextLine();

        System.out.println("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Hello " + name + ", age " + age);
        sc.close();
    }
}
```

Figure 1.17 Scanner Input Example



Control Flow Overview

Control Flow Structures

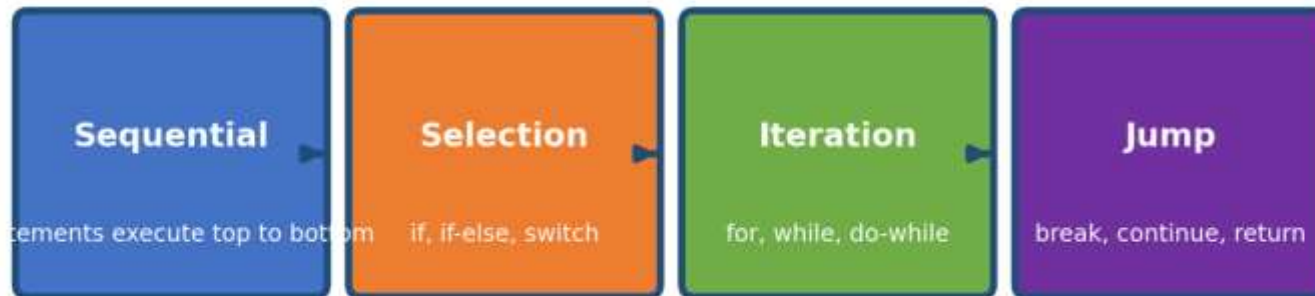


Figure 1.18 Control Flow Structures



Decision Making with if-else

- The if statement executes a block only when its boolean condition evaluates to true.
- else if chains test multiple conditions in order from top to bottom.
- Curly braces { } define blocks; omitting them works for single statements but is risky.
- Nested if statements handle complex decisions but should be kept readable with clear logic.

```
IFElseDemo.java

public class IFElseDemo {
    public static void main(String[] args) {
        int marks = 72;

        if (marks >= 90) {
            System.out.println("Grade: A");
        } else if (marks >= 75) {
            System.out.println("Grade: B");
        } else if (marks >= 60) {
            System.out.println("Grade: C");
        } else {
            System.out.println("Grade: Fail");
        }
    }
}
```

Figure 1.19 if-else Example



Multi-way Selection with switch

- The switch statement selects one branch among many based on an integer, char, or String value.
- Each case label must be a constant; duplicate case values cause a compile-time error.
- The break statement exits the switch; without it execution falls through to the next case.
- The default case handles values that match no case label, similar to a final else.
- Switch is cleaner than long if-else chains when comparing one variable to many constants.
- Java 14+ supports switch expressions that return a value using the arrow syntax (case 1 ->).



Loops: for, while, and do-while

- The for loop is ideal when the number of iterations is known (for (int i = 0; i < n; i++)).
- The while loop repeats while a condition is true and may execute zero times.
- The do-while loop always executes the body at least once before checking the condition.
- An infinite loop (while (true)) runs until broken by break or return inside the body.

```
LoopsDemo.java

public class LoopsDemo {
    public static void main(String[] args) {
        // for loop - known iterations
        for (int i = 1; i <= 5; i++) {
            System.out.println(i + " ");
        }
        System.out.println();

        // while loop - condition checked first
        int count = 0;
        while (count < 3) {
            System.out.println("While: " + count);
            count++;
        }

        // do-while - executes at least once
        int n = 0;
        do {
            System.out.println("Do-while: " + n);
            n++;
        } while (n < 2);
    }
}
```

Figure 1.21 Loop Examples



Building Complete Console Programs

- Combine variables, operators, Scanner input, and control flow to solve practical problems.
- Typical structure: import packages, declare class, main method, input, process, output.
- Validate user input with if conditions before using values in calculations.
- Use loops to process lists of numbers, generate tables, or repeat menu-driven actions.
- Format output with println and string concatenation for readable results.
- Practice programs (grade calculator, menu system, number series) reinforce all Chapter 1 topics.



References

1. Herbert Schildt, Java: The Complete Reference
Cay S. Horstmann, Core Java
Kathy Sierra & Bert Bates, Head First Java

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

